

EducationQ hoạt động như thế nào?

1. EducationQ thực sự đo điều gì?

EducationQ là framework mô phỏng quá trình dạy–học giữa các LLM để đánh giá **khả năng giảng dạy của Teacher LLM**. Nó không chỉ hỏi mô hình nào tự trả lời tốt hơn, mà hỏi:

Sau khi một Student LLM được một Teacher LLM hướng dẫn qua nhiều lượt, kết quả làm bài của Student thay đổi thế nào và quá trình hướng dẫn có chất lượng sư phạm ra sao?

Một thí nghiệm hoàn chỉnh được điều phối trong `main()`:

```
Dataset + YAML
  ↓
Student.take_test() → pre-test response + correctness
  ↓
Teacher.generate_question()
  ↳ lặp N vòng qua interaction_history
Student.answer_question()
  ↓
Student.take_test(history) → post-test
  ↓
Exact-match learning gain
  ↳ tùy chọn: pairwise LLM qualitative evaluation
```

EducationQ **không fine-tune** hay cập nhật trọng số của Student. Sự “học” diễn ra trong context: prompt post-test chứa câu trả lời cũ và lịch sử trao đổi với Teacher. Learning gain vì vậy đo thay đổi hành vi trả lời sau một phiên hướng dẫn có context, không chứng minh Student lưu kiến thức bền vững ngoài phiên đó.

2. Kiến trúc: ba agent và một bộ điều phối

Thành phần	Trách nhiệm trong code
TeacherLLM	Đọc kết quả pre-test và lịch sử để sinh một câu hỏi hoặc feedback mới mỗi vòng.
StudentLLM	Làm pre-test/post-test và trả lời Teacher trong giai đoạn tương tác.
EvaluatorLLM	Tính accuracy bằng exact match; trong mode chuyên biệt, gọi LLM để đánh giá định tính theo cặp Teacher.
EvalManager	Tải dữ liệu, xác định thứ tự gọi agent, chạy song song, lưu trạng thái và kết quả.

Ba lớp agent kế thừa `BaseLLM`. Lớp này tạo OpenAI-compatible client và gọi `chat.completions.create()` với `model`, `messages`, `temperature` và `max_tokens`. Trường `provider` có thể được chuyển qua `extra_body` để định tuyến provider trên OpenRouter.

Decorator `retry_api_call()` retry API theo exponential backoff. Khi hết số lần thử, nó log lỗi và trả `None`.

Đây không phải ba agent tự do trò chuyện với nhau. `EvalManager` quyết định agent nào được gọi, input là gì và output đi tới đâu. Trong interaction, Teacher và Student là hai API call tách biệt; Evaluator không đứng giữa để sửa hội thoại.

3. Entry point, cấu hình và dữ liệu

3.1. Khởi chạy

`main()` parse CLI, đọc YAML qua `EvalConfig.from_yaml()`, tạo output directory, thiết lập logging và khởi tạo `EvalManager`.

Mode	Luồng thực thi
<code>complete</code>	pre-test → interaction → post-test → quantitative evaluation
<code>pretest</code>	Chỉ chạy <code>_run_pretest()</code>
<code>load_pretest</code>	Đọc JSON → interaction → post-test → evaluation
<code>load_interaction</code>	Đọc JSON → post-test → evaluation
<code>evaluation</code>	Chạy một phép đánh giá định tính chuyên biệt

Trong constructor, `EvalManager` tạo mọi Teacher, mọi Student, một Evaluator, rồi gọi `_load_datasets()`.

3.2. Chuẩn hóa dataset

`_load_datasets()` chọn loader cho MMLU-Pro, GPQA hoặc AGIEval, sau đó lọc theo ID, category, số câu đầu tiên hoặc random sample. Một câu hỏi sau preprocessing có dạng khái niệm:

```
{
  "question_id": "10822",
  "question": "...",
  "options": ["...", "..."],
  "answer": "B",
  "answer_index": 1,
  "cot_content": "...",
  "category": "physics"
}
```

Tên trường đáp án có thể là `answer` hoặc `correct_answer`. `StudentLLM.take_test()` chuẩn hóa result bằng cách ưu tiên `correct_answer`, nếu không có thì dùng `answer`; answer index được xử lý tương tự.

`cot_content` trong validation data chỉ được đưa vào prompt khi agent có `use_few_shot=true`. Nó không phải lịch sử học tập sinh ra trong thí nghiệm.

4. Giai đoạn 1 — Pre-test

Pre-test tạo baseline cho từng Student trước khi gặp Teacher. Luồng này nằm trong `EvalManager._run_pretest()`.

4.1. Cách thực thi

Với mỗi Student, manager submit riêng từng question vào `ThreadPoolExecutor`:

```
student.take_test([question], few_shot_cot_examples)
```

Không truyền `interaction_history` hay `pre_test_results`, nên đây là lần làm bài trước dạy học. Sau mỗi task, manager gọi `evaluator.calculate_accuracy(result)` và lưu response cùng score.

Code kiểm tra cả question bị thiếu và question có `responses` rỗng. Chúng được chạy lại tối đa theo `student.answer_retries`. Nếu vẫn thất bại, framework tạo result rỗng kèm lỗi thay vì âm thầm bỏ câu.

4.2. Prompt làm bài

Trong `StudentLLM.take_test()`, system prompt yêu cầu Student:

- Trả lời câu hỏi trắc nghiệm thuộc một category.
- Suy luận từng bước.
- Kết thúc bằng `the answer is (X)`.

Nếu `use_few_shot=true`, các ví dụ được nối vào system prompt bằng `format_question()`, gồm question, options và `cot_content`. Question cần làm nằm ở user message cuối.

4.3. Parse đáp án và tính đúng/sai

`extract_answer()` không dùng LLM khác để đọc đáp án. Nó lần lượt:

1. Tìm `answer is (X)` hoặc `answer is X`.
2. Tìm `Answer: X`.
3. Lấy ký tự A–J cuối cùng còn phù hợp.
4. Nếu không tìm được, trả `"R"`; giá trị này sẽ bị tính sai.

Mỗi response giữ cả văn bản gốc và prediction đã parse:

```
{
  "question_id": "10822",
  "question": "...",
  "options": [...],
  "correct_answer": "C",
  "model_response": "... the answer is (B)",
  "model_prediction": "B",
  "category": "physics"
}
```

`EvaluatorLLM.calculate_accuracy()` chỉ exact-match `correct_answer` với `model_prediction` rồi tính accuracy theo category và overall. Dù nằm trong lớp `EvaluatorLLM`, method này là logic Python xác định và **không gọi evaluator model**.

5. Giai đoạn 2 — Teacher–Student interaction

`EvalManager._run_interactions()` tạo episode cho mọi tổ hợp Teacher × Student × question. Mỗi question được xử lý bởi `_process_question()`, còn các question độc lập chạy song song.

Với một question, code khởi tạo `interaction_history = []` rồi lặp `NUM_INTERACTIONS` vòng:

```
1. teacher.generate_question(pre-test, history, round)
2. student.answer_question(teacher_question, history, pre-test)
3. history.append({"question": ..., "answer": ...})
```

Ở vòng tiếp theo, cả hai agent nhận toàn bộ history. Response hiện tại trở thành context cho hành vi tương lai của Teacher và Student. Đây là cơ chế code-level tạo “dynamic educational scenario”; không có memory database hoặc hidden student state khác.

5.1. Prompt của Teacher

`TeacherLLM.generate_question()` dựng prompt từ bốn phần.

Vai trò và mục tiêu

System message định danh model là giáo viên chuyên gia trong category và yêu cầu:

- Phân tích response pre-test.
- Sinh đúng một câu hỏi mới mỗi lần gọi.
- Khi cần, sửa hoặc feedback câu trả lời vòng trước.
- Không tiết lộ đáp án đúng hoặc chi tiết cụ thể của pre-test.
- Phát triển hiểu biết để chuẩn bị cho post-test.

Nếu `recommended_education_theory` được cấu hình, tên lý thuyết được chèn vào role. Code không triển khai riêng một thuật toán ZPD, Bloom hay scaffolding; các chiến lược này được gọi ra bằng

instruction và context.

Few-shot examples

Khi `teacher.use_few_shot` bật, prompt thêm các cặp `Question – Reasoning` từ validation data, giới hạn bởi `num_if_few_shots`.

Thông tin pre-test

Với mỗi response pre-test, Teacher thấy:

- Question ID và nội dung câu hỏi.
- Lập luận nguyên văn của Student (`model_response`).
- Đáp án Student chọn (`model_prediction`).
- Nhãn `Correct` hoặc `Incorrect`.

Teacher **không được truyền trực tiếp trường** `correct_answer` trong prompt. Tuy nhiên, Teacher vẫn thấy question, response và đúng/sai nên có thể tự giải hoặc suy luận misconception. Phát biểu chính xác là “prompt không cung cấp trực tiếp đáp án chuẩn”, không phải “Teacher không thể biết đáp án”.

Lịch sử và yêu cầu vòng hiện tại

History được dựng thành:

```
system:    role + few-shot + pre-test info
assistant: Teacher: <câu hỏi vòng 1>
user:      Student: <câu trả lời vòng 1>
assistant: Teacher: <câu hỏi vòng 2>
user:      Student: <câu trả lời vòng 2>
...
user:      Generate the round k question ...
```

Teacher ở role `assistant` và Student ở role `user` vì API đang yêu cầu Teacher tiếp tục hội thoại. Khả năng điều chỉnh theo response là hành vi model sinh từ context, không phải rule engine chọn sẵn lesson plan.

5.2. Prompt của Student trong interaction

`StudentLLM.answer_question()` cũng dựng lại toàn bộ context thay vì duy trì server-side chat session.

System prompt yêu cầu Student phân tích, giải thích suy nghĩ, vận dụng khái niệm và nói rõ sự không chắc chắn. Nếu `include_pretest_info=true`, Student được xem lại question, options và **chính response pre-test của mình**. Khác Teacher, Student không nhận nhãn correct/incorrect.

History dùng role ngược lại:

```
user:      Teacher: <câu hỏi cũ>
assistant: Student: <câu trả lời cũ>
...
user:      Teacher: <câu hỏi mới>
           Your thoughtful and detailed answer ...
```

Student model sinh lượt assistant kế tiếp. Few-shot chỉ được nối nếu cấu hình yêu cầu.

5.3. Kiểm soát độ dài

`generate_question()` và `answer_question()` đếm output bằng `tiktoken`. Response hợp lệ khi token count lớn hơn 0 và không vượt:

```
max(max_tokens × max_tokens_rerun_threshold_percentage,
    recommended_token_limit)
```

Do đó, `recommended_*_token_limit` xuất hiện trong lời nhắc nhưng ngưỡng retry thực tế có thể lớn hơn nếu phần trăm của `max_tokens` lớn hơn. Hết retry, hai method log lỗi nhưng vẫn trả response cuối; comment nói trả chuỗi rỗng không phản ánh hành vi code hiện tại.

5.4. Resume

Sau một cặp Teacher–Student, `_run_interactions()` lưu:

- `progress_<version>.json`: đánh dấu pair hoàn thành.
- `progress_results_<version>.json`: hội thoại tích lũy.

Lần chạy sau bỏ qua pair đã hoàn tất. Checkpoint ở cặp Teacher–Student pair, không phải sau từng interaction round.

6. Giai đoạn 3 — Post-test

`EvalManager._run_posttest()` gọi lại:

```
student.take_test(
    pre_test_result,
    few_shot_cot_examples,
    interaction_history,
    pre_test_result,
)
```

Tham số đầu là danh sách response pre-test, vẫn chứa question, options và đáp án chuẩn để scoring. Tuy nhiên, `take_test()` không đưa `correct_answer` vào message.

So với pre-test, prompt có thêm:

1. Nếu `include_pretest_info=true`, question và response trước của Student.
2. Toàn bộ `interaction_history` dưới dạng các lượt Teacher–Student.
3. Chỉ dẫn suy nghĩ lại dựa trên hiểu biết sau cuộc trò chuyện.

Vì thế post-test **không phải closed-book retest độc lập**; nó chủ ý cho Student truy cập teaching context.

Output cho phép truy vết toàn episode:

```
teacher_name
├── student_name
│   ├── question_id
│   │   ├── category
│   │   ├── pre_test.responses/scores
│   │   ├── interaction[{question, answer}, ...]
│   └── post_test.responses/scores
```

7. Giai đoạn 4 — Hai lớp đánh giá khác nhau

7.1. Định lượng trong pipeline `complete`

`EvalManager._perform_evaluation()` không gọi evaluator model. Với mỗi Teacher–Student, nó đếm số câu đúng trước/sau theo category và overall:

```
pre_test_accuracy = pre_test_correct / total
post_test_accuracy = post_test_correct / total
progress          = post_test_accuracy - pre_test_accuracy
```

`progress` là Absolute Learning Gain. Khi nhiều Teacher dạy cùng một Student trên cùng tập câu hỏi, chênh lệch gain là tín hiệu định lượng về teaching effectiveness.

Pipeline `complete` kết thúc ở đây; nó **không tự động chạy** đánh giá định tính bên dưới.

7.2. Định tính trong mode `evaluation`

Mode này cần post-test JSON, CSV gồm các dòng `(question_id, teacher_1, teacher_2)` và `--eval-type`. Code hiện dùng Student đầu tiên trong `STUDENT_CONFIGS` để so sánh pairwise hai Teacher trên cùng question.

Toàn bộ interaction

`over_interaction_analysis()` nhận pre-test và hai hội thoại rồi chấm:

1. Assessment Effectiveness.

2. Questioning Effectiveness.
3. Feedback Effectiveness.
4. Instructional Adaptation Effectiveness.
5. Learning Objective Achievement Effectiveness.

Chỉ câu hỏi của Teacher

`teacher_questions_analysis()` chỉ format câu hỏi Teacher và chấm:

1. Question Relevance.
2. Cognitive Level.
3. Knowledge Dimension.
4. Question Diversity.
5. Scaffolding Progression.
6. Metacognitive Promotion.

Chỉ response của Student

`student_responses_analysis()` chỉ format câu trả lời Student dưới mỗi Teacher và chấm:

1. Response Relevance.
2. Cognitive Level Demonstration.
3. Knowledge Dimension Integration.
4. Response Diversity.
5. Elaboration Progression.
6. Metacognitive Reflection.

Mỗi dimension bắt buộc có `analysis` và `score`; verdict có `analysis` và choice A/B/C.

`create_dimension_schema()` tạo JSON Schema, còn `EvaluatorLLM.generate_response()` gửi schema qua `response_format` với `strict: true`.

Trước khi dựng prompt, code randomize Teacher thật vào `teacher_a` / `teacher_b`. Sau khi nhận output, `deanonymize_evaluation()` đổi key và verdict về tên thật (`C` thành `Tie`). Điều này giảm position bias A/B nhưng không loại bỏ các bias khác của LLM judge.

`comprehensive` gọi cả ba phép phân tích. Mỗi method randomize độc lập, nên ba analysis trong cùng task không nhất thiết dùng cùng thứ tự ẩn danh.

8. Dữ liệu đi qua toàn pipeline

```
Raw question
├─ Student.take_test()
│   └─ raw response + parsed prediction + exact-match score
├─ Teacher.generate_question(pre-test, history)
│   └─ question / feedback
├─ Student.answer_question(question, pre-test, history)
│   └─ student response
├─ history.append(question, answer) × N
├─ Student.take_test(pre-test, history)
│   └─ post-test response + prediction + score
├─ _perform_evaluation()
│   └─ accuracy trước/sau + progress
└─ optional pairwise Evaluator LLM
    └─ dimension analyses + scores + verdict
```

Mỗi giai đoạn lưu JSON bằng `_save_results()`, cho phép kiểm tra lại episode và tiếp tục từ một số giai đoạn mà không chạy lại toàn bộ API calls.

9. Những giới hạn và chi tiết phải đọc đúng

9.1. Learning gain phụ thuộc context

Post-test cho Student xem lại pre-test response và hội thoại. Gain có thể đến từ hiểu tốt hơn, nhớ gợi ý, tự sửa đáp án hoặc bất tín hiệu từ Teacher. Metric không tự phân tách các nguyên nhân này.

9.2. Không có verifier chống lộ đáp án

Prompt yêu cầu Teacher không tiết lộ đáp án, nhưng `_process_question()` không kiểm tra compliance trước khi chuyển output sang Student. Nếu Teacher vi phạm, hội thoại vẫn tiếp tục.

9.3. Scaffolding là hành vi emergent

Framework cung cấp diagnosis, history và instruction sự phạm, nhưng không có state machine bảo đảm câu hỏi tăng dần độ khó. Chất lượng scaffolding là thứ được đo, không phải thuộc tính code bảo đảm.

9.4. Evaluator có hai nghĩa

- `calculate_accuracy()` là deterministic Python scoring.
- Ba method `*_analysis()` mới gọi evaluator model làm LLM judge.

Không phân biệt chúng sẽ dẫn tới mô tả sai rằng mọi accuracy đều do GPT-4o chấm.

9.5. Không phải mọi trường YAML đều điều khiển runtime

`EvalConfig` không đọc `API_CALL_MAX_RETRIES`, `API_CALL_RETRY_DELAY` hay `PARALLEL_TASKS` từ YAML. Decorator và executor tham chiếu global `CONFIG`. Vì vậy, thay các trường cùng tên trong `config_template.yaml` không nhất thiết đổi retry/parallelism của runtime hiện tại.

9.6. Resume có nhiều mức

CLI cho phép tiếp tục từ pre-test JSON hoặc interaction JSON. Riêng `_run_interactions()` có progress file tự động ở cấp Teacher–Student pair; một pair dừng giữa chừng có thể phải chạy lại phần chưa được đánh dấu hoàn tất.

10. Kết luận

EducationQ biến đánh giá Teacher LLM thành một thí nghiệm theo thời gian:

1. Đo Student trước khi được dạy.
2. Cho Teacher quan sát lỗi và reasoning để hướng dẫn nhiều lượt.
3. Cho cùng Student làm lại câu hỏi với teaching context.
4. Đo thay đổi accuracy và, khi được yêu cầu, dùng Evaluator LLM phân tích chất lượng sự phạm theo cặp Teacher.

Cơ chế trung tâm là **prompt construction + truyền lịch sử + so sánh pre/post-test**. `EvalManager` tạo quy trình có kiểm soát; Teacher và Student tạo teaching episode; exact-match scoring đo learning gain; Evaluator LLM bổ sung đánh giá định tính. Vì vậy, EducationQ đo mức cải thiện của Student **trong một kịch bản in-context được mô phỏng**, rồi dùng mức cải thiện đó làm proxy cho năng lực giảng dạy của Teacher.